理解 KV 缓存

Understanding KV Cache

原文作者：@techNmak。

先列专业术语对照表，再提供逐页完整中文译文。每个原文页面的译文之后附原文矢量对照页，完整保留公式、图表、代码和英文引用，便于核对。

正文中文：LXGW WenKai（霞鹜文楷）；英文与数字：Times New Roman。原文对照页保留原始字体。

本文采用翻译辅助服务生成初稿，统一技术术语并检查结构、字体和导航；涉及精确数值、公式或实现细节时，可直接对照相邻原文页。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Inference	推理	
KV Cache	键值缓存	KV Cache
PagedAttention	PagedAttention（方法名保留）	
Head Dimension	头维度	
Quantization	量化	
Transformer	Transformer 模型	Transformer
Attention	注意力机制	Attention
KV Heads	键值头	
Serving	推理服务	
Values	值	
Tokens	词元	
Keys	键	
Causal Mask	因果掩码	
Generation	生成	
Decoder	解码器	
Prefill	预填充	
Token	词元	
Model	模型	
Learned Projections	学习得到的投影	
Self-attention	自注意力	
Representation	表示	
Query	查询	Q
Value	值	V
Key	键	K
Key/Value Heads	键值头	KV Head
Mental Model	理解框架	
Batch Size	批大小	
Kernels	内核	
Matrix	矩阵	
Dense Attention	稠密注意力	
Query Length	查询长度	L
Attention Weights	注意力权重	

英文原词	中文译名	缩写／符号
Tensor	张量	
Models	模型	
Incremental Decoding	增量解码	
Memory Traffic	内存流量	
Resident	驻留	
Kernel	内核	
Multi-head Attention	多头注意力	MHA
Query Heads	查询头	
Value Heads	值头	
Query Head	查询头	
Key Heads	键头	
Baseline	基线	
Multi-query Attention	多查询注意力	MQA
Grouped-query Attention	分组查询注意力	GQA
Heads	头	
Multi-head Latent Attention	多头潜在注意力	MLA
Projection Absorption	投影吸收	
Latent Compression	潜在压缩	
Vector	向量	
Decoupled Positional Component	解耦的位置分量	
Rotary Position Embeddings	旋转位置嵌入	RoPE
Queries	查询	
Cache Capacity	缓存容量	
Fragmentation	碎片化	
FlashAttention	FlashAttention（方法名保留）	
Packing	打包	
Latency	延迟	
Data Movement	数据移动	
Host Memory	主机内存	
Calibration	校准	
Activations	激活	
Throughput	吞吐量	
Benchmark	基准测试	
Precision	精度	

英文原词	中文译名	缩写／符号
Accuracy	准确性	
Quantization Error	量化误差	
Activation	激活	
Efficient Attention	高效注意力	
Memory Bandwidth	内存带宽	
Recomputation	重计算	
Numerical Precision	数值精度	
Causal Attention	因果注意力	
Large Language Model	大语言模型	LLM
Mixture-of-Experts	混合专家	MoE
Quantized	量化后的	
Training	训练	
Prefix Caching	前缀缓存	
Offloading	卸载	
Eviction	逐出	
Block Table	块表	
Block Sharing	块共享	
Cache Hits	缓存命中	
Cache Key	缓存键	
Least Recently Used	最近最少使用	LRU
Layer State	层状态	
Tensor Payload	张量有效载荷	
Trust Boundary	信任边界	
Low-rank Latent Compression	低秩潜在压缩	

目录导航

点击章节标题跳转到中文译文，点击“原文”核对公式和图表。页码按本译本最终分页生成。

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

理解 KV 缓存 · 第 6 页 ［原文 1］

阅读导引 · 第 8 页 ［原文 2］

1 — 自回归生成 · 第 10 页 ［原文 3］

2 — 冗余计算 · 第 12 页 ［原文 4］

3 — 缓存机制 · 第 14 页 ［原文 5］

4 — 每个注意力层各有缓存 · 第 16 页 ［原文 6］

5 — 预填充与解码 · 第 18 页 ［原文 7］

6 — 逐步解码 · 第 20 页 ［原文 8］

7 — 内存容量公式 · 第 22 页 ［原文 9］

8 — 内存容量计算示例 · 第 24 页 ［原文 10］

9 — 上下文长度 · 第 26 页 ［原文 11］

10 — KV 缓存节省的资源 · 第 28 页 ［原文 12］

11 — 多头注意力（MHA） · 第 30 页 ［原文 13］

12 — 多查询注意力（MQA） · 第 32 页 ［原文 14］

13 — 分组查询注意力（GQA） · 第 34 页 ［原文 15］

14 — 一个公式，三种架构 · 第 36 页 ［原文 16］

15 — 多头潜在注意力（MLA） · 第 38 页 ［原文 17］

16 — 位置信息 · 第 40 页 ［原文 18］

17 — 从单个请求到服务器 · 第 42 页 ［原文 19］

18 — 碎片化 · 第 44 页 ［原文 20］

19 — PagedAttention · 第 46 页 ［原文 21］

20 — 块大小 · 第 48 页 ［原文 22］

21 — 前缀缓存 · 第 50 页 ［原文 23］

22 — 前缀缓存的限制 · 第 52 页 ［原文 24］

23 — 前缀缓存安全 · 第 54 页 ［原文 25］

24 — 缓存逐出 · 第 56 页 ［原文 26］

25 — 缓存卸载 · 第 58 页 ［原文 27］

26 — KV 缓存量化 · 第 60 页 ［原文 28］

27 — 量化并非没有代价 · 第 62 页 ［原文 29］

28 — 常见误解 · 第 64 页 ［原文 30］

29 — 生产环境中的理解框架 · 第 66 页 ［原文 31］

30 — 完整理解框架 · 第 68 页 ［原文 32］

参考文献 · 第 70 页 ［原文 33］

理解 KV 缓存

对应原文第 1 页；点击查看相邻原文对照页

理解键值缓存（KV Cache）

从自回归解码（Autoregressive Decoding）到现代大语言模型推理（LLM Inference）

面向 AI 工程师、开发者及深入学习者的技术手册。

作者：@techNmak。理解 AI 系列（Understanding AI Series），手册 02。

Understanding KV Cache

From Autoregressive Decoding to Modern LLM Inference

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

Handbook 02 - Understanding AI Series

阅读导引

对应原文第 2 页；点击查看相邻原文对照页

本手册将澄清哪些问题

键值缓存（KV Cache）很容易被描述得不准确。人们常把它称为“大语言模型的记忆”“加速技巧”，或简单称为“缓存的注意力”。这些简略说法掩盖了真正重要的机制。

本手册从基本原理出发解释这一概念。读完后，你应能说明：

- 为什么普通因果 Transformer 中过去的键（Keys）和值（Values）可以复用；

- 究竟存储了哪些张量（Tensors），其形状如何增长；

- 为什么缓存内存随缓存词元（Tokens）数、层数、键值头（KV Heads）数、头维度（Head Dimension）和每个元素的字节数线性增长；

- 为什么 MQA、GQA、MLA 通过不同方式降低 KV 成本；

- 为什么并发推理服务（Serving）中的缓存管理会成为系统问题；

- 分页注意力（PagedAttention）、前缀缓存（Prefix Caching）、卸载（Offloading）及 KV 缓存量化（KV-cache Quantization）实际改变了什么。

先备知识：你应已了解基本的注意力（Attention）公式，以及 Q、K、V 的作用。前一本《理解注意力》（Understanding Attention）是自然的先修材料。

目标不是再记住一种优化，而是理解其数学有效性、节省的资源以及仍然存在的成本。

研究依据：[1]、[2]、[5]、[6]。

ORIENTATION

What this handbook will make clear

A KV cache is easy to describe badly. It is often called “LLM memory,” a “speed trick,” or simply “cached attention.” Those shortcuts hide the mechanism that matters.

This handbook builds the idea from first principles. By the end, you should be able to explain:

- why past keys and values are reusable in an ordinary causal Transformer;
- exactly what tensors are stored and how their shapes grow;
- why cache memory scales linearly with cached tokens, layers, KV heads, head dimension, and bytes per element;
- why MQA, GQA, and MLA attack KV cost in different ways;
- why cache management becomes a systems problem under concurrent serving;
- what PagedAttention, prefix caching, offloading, and KV-cache quantization actually change.

Prerequisites

You should already know the basic attention equation and the role of Q, K, and V. The previous handbook, *Understanding Attention*, is the natural prerequisite.

The goal is not to memorize another optimization. It is to understand why the optimization is mathematically valid, what resource it saves, and what costs remain.

Research anchors: [1], [2], [5], [6]

1 — 自回归生成

对应原文第 3 页；点击查看相邻原文对照页

为什么解码是顺序执行的

仅解码器语言模型（Decoder-only Language Model）根据已有词元（Tokens），为下一个词元（Token）给出条件概率分布，原文公式为 $p(x_{t+1} \mid x_{1:t})$。

选定一个词元后，它被追加到序列，成为下一步上下文的一部分。因此，生成（Generation）形成依赖链：已有序列 → 下一个词元 → 再下一个词元 → ……完整符号见原文对照页。

两个执行阶段：对于包含多个词元的提示（Prompt），模型可以在因果掩码（Causal Mask）约束下同时处理这些提示位置，这通常称为预填充（Prefill）。生成开始后，每个解码步骤通常让每条序列向前推进一个新生成的词元，这就是解码（Decode）阶段。

图示标签：Prompt＝提示；Prefill＝预填充；Decode＝解码。图中依次显示提示、预填充及连续的解码步骤。

正是在这种顺序解码结构中，KV 缓存变得至关重要。

研究依据：[1]、[6]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

1 - AUTOREGRESSIVE GENERATION

Why decoding is sequential

A decoder-only language model assigns a distribution to the next token conditioned on the tokens already available:

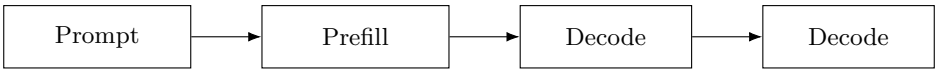
$$p(x_{t+1} \mid x_{1:t}).$$

After a token is selected, it is appended to the sequence and becomes part of the context for the next step. Generation therefore forms a dependency chain:

$$x_1, \ldots, x_t \to x_{t+1} \to x_{t+2} \to \cdots$$

Two execution phases

For a prompt of many tokens, the model can process those prompt positions together under a causal mask. This is usually called **prefill**. Once generation begins, the model usually advances one newly generated token per sequence per decode step. This is the **decode** phase.



This sequential decode structure is the setting in which KV caching becomes crucial.

Research anchors: [1], [6]

2 — 冗余计算

对应原文第 4 页；点击查看相邻原文对照页

朴素解码器会重复哪些工作

考虑一个自注意力（Self-attention）层。该层对每个词元表示（Token Representation）计算学习得到的投影（Learned Projections），形成查询 q、键 k、值 v；三个投影公式保留在原文对照页。

在解码步骤 t，新查询（Query）必须关注位置 1 至 t 的键（Keys）和值（Values）。朴素实现可能每一步都从头重建过去的 k、v 张量，重复计算输入没有变化的投影。

关键观察：对于普通因果 Transformer，一旦位置 i 在某一固定层处理完毕，追加未来词元不会反过来改变该位置的层状态（Layer State）。因果掩码阻止未来位置的信息回流到较早词元的表示中。

因此，较早位置产生的键和值可以保留，并在后续解码步骤复用。

KV 缓存之所以有效，是因为在模型的因果计算下，过去的层状态不随未来词元变化。这不是启发式猜测。

研究依据：[1]、[6]。

2 - THE REDUNDANT COMPUTATION

What a naive decoder would repeat

Consider one self-attention layer. For every token representation, the layer forms learned projections:

$$q_i = h_i W_Q, \qquad k_i = h_i W_K, \qquad v_i = h_i W_V.$$

At decode step t, the new query must attend to the keys and values of positions $1, \ldots, t$.

A naive implementation could rebuild the old k_i and v_i tensors from scratch on every step. That would repeatedly perform projections whose inputs have not changed.

The useful observation

For an ordinary causal Transformer, once position i has been processed at a fixed layer, adding future tokens does not retroactively change that position's layer state. Causal masking prevents information from future positions from flowing backward into the representation of an earlier token.

Therefore the key and value derived for an earlier position can be retained and reused on later decode steps.

KV caching is valid because past layer states are invariant to future tokens under the model's causal computation. It is not a heuristic guess.

Research anchors: [1], [6]

3 — 缓存机制

对应原文第 5 页；点击查看相邻原文对照页

究竟缓存了什么

假设模型已处理位置 1 至 t-1。在一个注意力（Attention）层中，缓存保存此前的键序列与值序列，分别记作 $K_{1:t-1}$ 和 $V_{1:t-1}$。

对于新位置 t，该层只计算新的查询、键和值，即 q_t、k_t、v_t，然后将新的键（Key）和值（Value）追加到已有缓存，得到 $K_{1:t}$、$V_{1:t}$。

当前查询对累积的历史执行注意力计算：将查询与历史键的转置相乘，按头维度的平方根缩放，经过 softmax，再与历史值相乘。完整分数、上下标和矩阵公式见紧接的原文对照页。

这一机制没有缓存什么：缓存不存储“想法”或符号化事实。它为每个注意力层、每个缓存词元位置存储数值化的键／值状态（Key/Value States）；物理布局由推理引擎（Inference Engine）决定。

研究依据：[1]、[6]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

3 - THE MECHANISM

What exactly is cached?

Suppose the model has already processed positions $1, \ldots, t-1$. At one attention layer, it has stored

$$K_{1:t-1} = [k_1; \ldots; k_{t-1}], \qquad V_{1:t-1} = [v_1; \ldots; v_{t-1}].$$

For the new position t, the layer computes only the new projections

$$q_t, \qquad k_t, \qquad v_t,$$

then appends the new key and value:

$$K_{1:t} = [K_{1:t-1}; k_t], \qquad V_{1:t} = [V_{1:t-1}; v_t].$$

The current query attends over the accumulated history:

$$\text{softmax}\left(\frac{q_t K_{1:t}^T}{\sqrt{d_h}}\right) V_{1:t}.$$

What is not cached by this idea

The cache does not store “thoughts” or symbolic facts. It stores numerical key/value states for each attention layer and cached token position, in whatever physical layout the inference engine uses.

Research anchors: [1], [6]

4 — 每个注意力层各有缓存

对应原文第 6 页；点击查看相邻原文对照页

缓存不是单个矩阵

每个自注意力（Self-attention）层都有自己的学习投影，因此也有自己的 K/V 状态。具有 L 个注意力层的模型，维护第 1 层的 K、V，第 2 层的 K、V，直至第 L 层的 K、V。

在常见逻辑布局下，每层的 K、V 张量（Tensors）形状都是 B × H_KV × T × D_h。原文中的实数域及上下标公式完整保留在对照页。

其中，B 是批大小（Batch Size）；H_KV 是键值头（Key/Value Heads）数；T 是缓存序列长度（Cached Sequence Length）；D_h 是头维度（Head Dimension）。

实际内核（Kernels）可能以不同方式存储或打包这些维度。逻辑形状是理解机制的关键，特定物理布局属于实现细节。

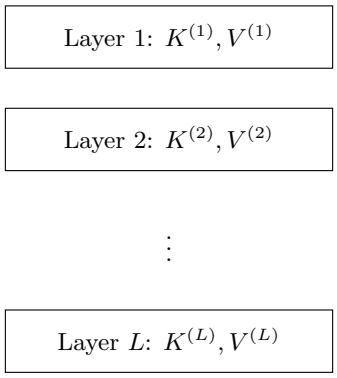
研究依据：[2]、[6]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

4 - ONE CACHE PER ATTENTION LAYER

The cache is not a single matrix

Each self-attention layer has its own learned projections and therefore its own K/V state. A model with L attention layers maintains a logical history like:



For a common logical layout, one layer's cache can be thought of as

$$K, V \in \mathbb{R}^{B \times H_{KV} \times T \times D_h},$$

where B is batch size, H_{KV} is the number of key/value heads, T is cached sequence length, and D_h is head dimension.

Actual kernels may store or pack dimensions differently. The logical shape is the important mental model; a particular physical layout is an implementation detail.

Research anchors: [2], [6]

5 — 预填充与解码

对应原文第 7 页；点击查看相邻原文对照页

两个阶段以不同方式创建缓存

预填充（Prefill）：给定输入提示，模型在因果掩码约束下，于每一层并行计算多个提示位置的 K/V 状态，填入初始缓存。

解码（Decode）：对于每个新生成的词元，每层计算新的 q_t、k_t、v_t，追加 k_t、v_t，然后让新查询针对缓存历史执行注意力计算。

原表完整中文对照：

- 新处理的位置：预填充处理多个提示词元；解码通常每条序列、每一步处理一个新词元。

- 缓存操作：预填充建立初始 K/V 历史；解码追加新的 K/V 状态。

- 注意力查询长度（Query Length）：预填充通常为多个位置；解码通常为 1。

- 复用收益：预填充建立状态；解码避免重算过去的 K/V 投影。

KV 缓存不会消除针对历史的注意力计算。在稠密注意力（Dense Attention）中，新查询仍须与过去的 K/V 状态交互。

研究依据：[6]、[7]。

5 - PREFILL AND DECODE

The cache is created differently in the two phases

Prefill

Given an input prompt, the model computes K/V states for many prompt positions in parallel at each layer, subject to causal masking. These states populate the initial cache.

Decode

For each newly generated token, each layer computes a new q_t, k_t, v_t, appends k_t, v_t, and performs attention for the new query against the cached history.

	Prefill	Decode
New positions processed	Many prompt tokens	Usually one new token per sequence/step
Cache action	Build initial K/V history	Append new K/V state
Attention query length	Many	Usually 1
Reuse benefit	Establishes state	Avoids recomputing past K/V projections

A KV cache does not eliminate attention over the history. The new query still has to interact with past K/V state for dense attention.

Research anchors: [6], [7]

6 — 逐步解码

对应原文第 8 页；点击查看相邻原文对照页

四个词元的例子

假设缓存目前保存位置 1 至 3。下一步执行：

1. 模型在这一层接收第 4 个词元的表示（Representation）。

2. 计算 q_4、k_4、v_4。

3. 将 k_4、v_4 追加到该层缓存。

4. 将 q_4 与 $K_{1:4}$ 比较。

5. 用得到的注意力权重（Attention Weights）对 $V_{1:4}$ 加权混合。

6. 该层输出表示，供模型其余部分使用。

过去的 k_1、k_2、k_3 及 v_1、v_2、v_3 被读取，而不是重新生成。

图示标签：Cached＝已缓存，new＝新增。已缓存的 $K_{1:3}$、$V_{1:3}$ 与新增 k_4、v_4 组合，形成 $K_{1:4}$、$V_{1:4}$。

研究依据：[6]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

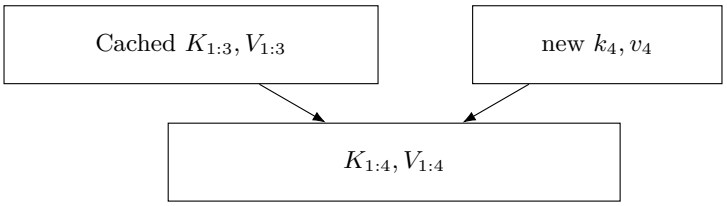
6 - STEP-BY-STEP DECODE

A four-token example

Imagine the cache currently holds positions 1–3. At the next step:

1. The model receives the representation for token 4 at this layer.
2. It computes q_4, k_4, and v_4.
3. It appends k_4 and v_4 to the layer cache.
4. q_4 is compared with $K_{1:4}$.
5. The resulting attention weights mix $V_{1:4}$.
6. The layer produces the representation used by the rest of the model.

The old k_1, k_2, k_3 and v_1, v_2, v_3 are read, not regenerated.



Research anchors: [6]

7 — 内存容量公式

对应原文第 9 页；点击查看相邻原文对照页

KV 缓存有多大

对于键值头数和头维度统一的传统稠密注意力（Dense Attention）堆栈，理想化的单序列缓存容量等于：$2 \times L \times T \times H_{KV} \times D_h \times b$。

- 系数 2 同时计入 K 和 V；

- L 是注意力层数；

- T 是已缓存词元数；

- H_{KV} 是键值头（Key/Value Heads）数；

- D_h 是头维度（Head Dimension）；

- b 是每个存储元素的字节数。

对于 B 条等长序列，再乘以 B。若各序列缓存长度 T⊠ 不同，则把 T 换成所有 T⊠ 的总和。原文的求和范围及完整数学排版见对照页。

实际推理服务引擎按块分配，还可能加入填充、缩放因子、元数据、分片及模型专有状态。因此，这些公式描述张量有效载荷（Tensor Payload），并不涵盖运行时消耗的每一个字节。

这个公式将架构选择与成本联系起来：上下文长度改变 T；MQA/GQA 改变 H_{KV}；缓存量化（Quantization）改变 b，同时增加量化元数据。

研究依据：[2]、[3]、[4]、[5]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

7 - THE MEMORY FORMULA

How large is the KV cache?

For a conventional dense attention stack with uniform KV-head count and head dimension, an idealized per-sequence cache size is

$$\boxed{M_{KV} = 2\,L\,T\,H_{KV}\,D_h\,b}$$

where:

- 2 accounts for both K and V;
- L is the number of attention layers;
- T is cached tokens;
- H_{KV} is the number of key/value heads;
- D_h is head dimension;
- b is bytes per stored element.

For B equal-length sequences, multiply by B. More generally, with variable cached lengths T_i, the idealized payload is

$$M_{KV} = 2\,L\,H_{KV}\,D_h\,b\sum_{i=1}^{B} T_i.$$

Real serving engines allocate in blocks and may add padding, scales, metadata, sharding, or model-specific states, so these formulas describe the tensor payload rather than every byte consumed by a runtime.

The formula is useful because each architectural choice has a visible lever: context length changes T; MQA/GQA change H_{KV}; cache quantization changes the storage term b (plus quantization metadata).

Research anchors: [2], [3], [4], [5]

8 — 内存容量计算示例

对应原文第 10 页；点击查看相邻原文对照页

同一模型中的 MHA、GQA 与 MQA

取一个示例解码器（Decoder）：L＝32 层，D_h＝128，b＝2 字节，查询头数 H_Q＝32。唯一改变的量是键值头数 H_KV。

原表中文对照，依次列出架构、键值头数、每词元 KiB、8,192 词元时的 GiB：

- 多头注意力（Multi-head Attention，MHA）：32；512 KiB；4.000 GiB。

- 分组查询注意力（Grouped-query Attention，GQA，示例配置）：8；128 KiB；1.000 GiB。

- 多查询注意力（Multi-query Attention，MQA）：1；16 KiB；0.125 GiB。

GQA 行的计算：$2 \times 32 \times 8192 \times 8 \times 128 \times 2 = 1{,}073{,}741{,}824$ 字节＝1 GiB。

这些是单序列张量有效载荷示例，不是所有 32 层模型的通用大小。模型的层数、键值头数、维度、数据类型（Dtypes）、局部注意力模式及分片方式可能不同。

研究依据：[2]、[3]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

8 - WORKED MEMORY EXAMPLE

MHA, GQA, and MQA in one model

Take an illustrative decoder with

$$L = 32, \quad D_h = 128, \quad b = 2 \text{ bytes}, \quad H_Q = 32.$$

The only value we change is H_{KV}.

Architecture	H_{KV}	KiB / token	GiB at 8,192 tokens
MHA	32	512	4.000
GQA (illustrative)	8	128	1.000
MQA	1	16	0.125

For the GQA row:

$$2 \cdot 32 \cdot 8192 \cdot 8 \cdot 128 \cdot 2 = 1{,}073{,}741{,}824 \text{ bytes} = 1 \text{ GiB}.$$

These are **per-sequence tensor-payload examples**, not universal sizes for all 32-layer models. Different models use different layer counts, KV heads, dimensions, dtypes, local-attention patterns, and sharding.

Research anchors: [2], [3]

9 — 上下文长度

对应原文第 11 页；点击查看相邻原文对照页

为什么长上下文改变推理服务成本

其他条件固定时，KV 缓存容量与 T 成正比。缓存词元数加倍，理想化的 KV 张量有效载荷也加倍。

沿用前页模型，原表中的 8K、32K、128K 分别对应三个上下文长度：

- MHA，H_KV＝32：4 GiB、16 GiB、64 GiB。

- GQA，H_KV＝8：1 GiB、4 GiB、16 GiB。

- MQA，H_KV＝1：0.125 GiB、0.5 GiB、2 GiB。

某个请求的上下文更长，不会让模型参数内存增长，却会让该请求的 KV 状态增长。因此，即使模型权重不变，长上下文能力也会带来推理服务容量问题。

上下文窗口（Context Window）规格告诉你模型能接受多长的输入，却不能告诉你某台服务器能够同时容纳多少个长上下文请求。

研究依据：[5]、[7]。

9 - CONTEXT LENGTH

Why long context changes serving economics

Holding everything else fixed,

$$M_{KV} \propto T.$$

That means doubling the number of cached tokens doubles the idealized KV tensor payload.

Using the same illustrative model as the previous page:

Architecture	8K	32K	128K
MHA ($H_{KV} = 32$)	4 GiB	16 GiB	64 GiB
GQA ($H_{KV} = 8$)	1 GiB	4 GiB	16 GiB
MQA ($H_{KV} = 1$)	0.125 GiB	0.5 GiB	2 GiB

A model's parameter memory does not grow just because one request has more context, but its per-request KV state does. This is why long-context capability creates a serving-capacity problem even when the model weights are unchanged.

A context-window specification tells you what the model can accept. It does not tell you how many long-context requests a particular server can hold concurrently.

Research anchors: [5], [7]

10 — KV 缓存节省的资源

对应原文第 12 页；点击查看相邻原文对照页

以及仍然存在的成本

KV 缓存避免重算历史键／值投影，以及产生这些投影所需的较早层状态，大幅减少自回归解码（Autoregressive Decoding）中的冗余工作。

但在普通稠密注意力（Dense Attention）中，新查询仍需访问过去的 K/V 状态。T 增长时，新位置的注意力计算仍会与更长的历史交互。

必须区分三种资源：

- 计算（Computation）：投影及注意力所需的算术运算。

- 容量（Capacity）：让 K/V 状态驻留所需的字节数。

- 内存流量／带宽（Memory Traffic/Bandwidth）：解码时为注意力内核提供数据而搬运的字节数。

MQA 的明确动机是降低增量解码（Incremental Decoding）中反复加载大 K/V 张量的内存带宽成本，而不只是“使用更少的参数”。

研究依据：[2]。

10 - WHAT KV CACHING SAVES

And what it does not save

KV caching avoids recomputing the historical key/value projections and the earlier layer states needed to produce them. That is a major reduction in redundant work during autoregressive decoding.

But for ordinary dense attention, the new query still needs access to past K/V state. As T grows, the attention operation for the new position still interacts with a longer history.

Three different resources

Do not collapse these into one claim:

- **Computation:** arithmetic required by projections and attention.
- **Capacity:** bytes needed to keep K/V state resident.
- **Memory traffic/bandwidth:** bytes moved to feed the attention kernel during decoding.

MQA was explicitly motivated by the memory-bandwidth cost of repeatedly loading large K/V tensors during incremental decoding, not merely by "using fewer parameters."

Research anchors: [2]

11 — 多头注意力（MHA）

对应原文第 13 页；点击查看相邻原文对照页

KV 状态成本较高的基线

在标准多头注意力（Multi-head Attention，MHA）中，每个查询头（Query Head）都有对应的键头（Key Head）和值头（Value Head）。概念上，H_KV＝H_Q。

如果有 32 个查询头，就有 32 个键头和 32 个值头。解码时，每个新查询头关注其对应头的历史 K/V 状态。这提供丰富的各头独立 K/V 投影，也使 KV 缓存随键值头数成正比增长。

图示中 Q_1 至 Q_6 分别对应 K_1/V_1 至 K_6/V_6。Q＝查询，K＝键，V＝值；同一编号属于同一头。

这一结构是理解 MQA 和 GQA 的比较基线（Baseline）。

研究依据：[1]、[2]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

11 - MULTI-HEAD ATTENTION

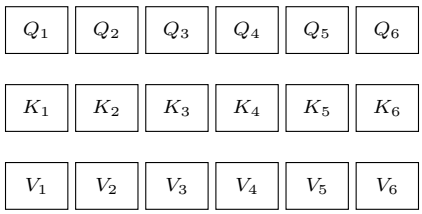
The expensive baseline for KV state

In standard multi-head attention, each query head has a corresponding key head and value head. Conceptually:

$$H_{KV} = H_Q.$$

If there are 32 query heads, there are 32 key heads and 32 value heads.

During decode, each new query head attends to the historical K/V state associated with its head. This gives the architecture rich per-head K/V projections, but it also makes the KV cache grow in direct proportion to the number of KV heads.



This page is the baseline against which MQA and GQA make sense.

Research anchors: [1], [2]

12 — 多查询注意力（MQA）

对应原文第 14 页；点击查看相邻原文对照页

在查询头之间共享 K/V

多查询注意力（Multi-query Attention，MQA）保留多个查询头，但这些头共享一个键头和一个值头：$H_Q > 1$，$H_KV = 1$。

Noam Shazeer 在 2019 年论文中专门从增量解码（Incremental Decoding）出发说明这一修改：加载 K/V 张量可能成为内存带宽瓶颈，缩小张量可以提高解码效率。

改变了什么：查询头仍各自独立。减少来自 K/V 共享，并非删除查询头。

缓存公式的含义：若一个有 32 个查询／键值头的 MHA 模型改为一个键值头，同时保持 L、D_h、T、b 不变，KV 张量有效载荷将变为原来的 1/32。

这是架构带来的算术结果。特定 MQA 模型相对特定 MHA 基线是否保持质量，是经验问题，不是数学保证。

研究依据：[2]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

12 - MULTI-QUERY ATTENTION

Share K/V across query heads

Multi-query attention (MQA) keeps multiple query heads but shares a single key head and a single value head across them:

$$H_Q > 1, \qquad H_{KV} = 1.$$

Noam Shazeer's 2019 paper motivates this change specifically around incremental decoding: loading K/V tensors can become a memory-bandwidth bottleneck, so shrinking those tensors can improve decoding efficiency.

What changes

The query heads remain distinct. The reduction comes from K/V sharing, not from deleting query heads.

What the cache formula says

If an MHA model with 32 query/KV heads moved to one KV head while keeping the same L, D_h, T, b, the KV tensor payload would be reduced by a factor of 32.

That arithmetic is an architectural consequence. Whether a particular MQA model preserves quality relative to a particular MHA baseline is an empirical question, not a mathematical guarantee.

Research anchors: [2]

13 — 分组查询注意力（GQA）

对应原文第 15 页；点击查看相邻原文对照页

MHA 与 MQA 之间的折中

分组查询注意力（Grouped-query Attention，GQA）的键值头少于查询头，但多于一个：1＜H_KV＜H_Q。查询头被划分成组，同组查询头共享同一个 K/V 头。

原表中文对照：

- 查询头数：MHA、GQA、MQA 都有多个查询头。

- 键值头数：MHA 与查询头数相同；GQA 介于两端；MQA 为 1。

- K/V 共享方式：MHA 的查询头之间不共享；GQA 在组内共享；MQA 在所有查询头之间共享。

Ainslie 等人将 GQA 作为 MQA 的推广。他们报告，经过追加训练（Uptraining）的 GQA 模型，在其试验中取得接近 MHA 的质量及与 MQA 相当的速度。

“GQA 与 MHA 一样好”过于宽泛。合理的表述应限定于 GQA 论文报告的模型和试验设置。

研究依据：[3]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

13 - GROUPED-QUERY ATTENTION

A middle point between MHA and MQA

Grouped-query attention (GQA) uses fewer KV heads than query heads, but more than one:

$$1 < H_{KV} < H_Q.$$

Query heads are partitioned into groups, and the query heads in a group share the same K/V head.

	MHA	GQA	MQA
Query heads	many	many	many
KV heads	same as Q	intermediate	1
K/V sharing	none across Q heads	within groups	across all Q heads

Ainslie et al. introduced GQA as a generalization of MQA and reported that their uptrained GQA models achieved quality close to MHA with speed comparable to MQA in their experiments.

"GQA is as good as MHA" is too broad. The defensible statement is about the reported models and experimental setup in the GQA paper.

Research anchors: [3]

14 — 一个公式，三种架构

对应原文第 16 页；点击查看相邻原文对照页

为什么键值头数如此重要

L、T、D_h、b 固定时，KV 缓存容量与 H_KV 成正比。因此，头共享方式直接影响内存消耗。

原图完整标签对照：Q＝查询头，KV＝键值头。MHA 的 4 个 Q 对应 4 个 KV；GQA 的 4 个 Q 分组共享 2 个 KV；MQA 的 4 个 Q 共享 1 个 KV。MHA、GQA、MQA 的名称分别为多头注意力、分组查询注意力、多查询注意力。

这就是为什么不能脱离注意力架构讨论 KV 缓存。参数规模及上下文长度相同的模型，也可能因为 K/V 表示（Representation）不同而有非常不同的缓存需求。

研究依据：[2]、[3]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

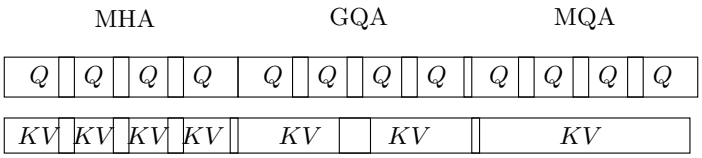
14 - ONE FORMULA, THREE ARCHITECTURES

Why KV-head count matters so much

For fixed L, T, D_h, b,

$$M_{KV} \propto H_{KV}.$$

So the head-sharing choice is visible directly in memory consumption.



This is why KV cache cannot be discussed independently of attention architecture. A model with the same parameter scale and context length can have very different cache requirements depending on its K/V representation.

Research anchors: [2], [3]

15 — 多头潜在注意力（MLA）

对应原文第 17 页；点击查看相邻原文对照页

压缩表示，而不只是共享头

DeepSeek-V2 提出多头潜在注意力（Multi-head Latent Attention，MLA），通过另一条路径降低 KV 缓存成本：用较低维的潜在状态表示不含位置信息的 K/V 信息。在推理时，投影吸收（Projection Absorption）使模型无需按传统方式显式生成并缓存完整的各头 K/V 张量。

这与 MQA/GQA 的概念不同：MQA 使用一个共享 K/V 头；GQA 使用较少的共享 K/V 头；MLA 对 K/V 信息进行低秩潜在压缩（Low-rank Latent Compression）。

DeepSeek-V2 报告，相对于其声明的比较架构，KV 缓存减少 93.3%。这是该论文的特定结果，不是通用的“MLA 压缩比”。

RoPE 带来的复杂性：DeepSeek-V2 还使用解耦的旋转位置分量（Decoupled Rotary Positional Components），因为普通旋转位置嵌入（RoPE）会与 MLA 的投影吸收策略相互作用。因此，“MLA 缓存一个很小的向量”这种概括并不完整。

研究依据：[4]。

15 - MULTI-HEAD LATENT ATTENTION

Compress the representation instead of only sharing heads

DeepSeek-V2 introduced Multi-head Latent Attention (MLA) as a different route to reducing KV-cache cost. Its central idea is to represent the non-positional K/V information through a lower-dimensional latent state. At inference, projection absorption lets the model avoid materializing and caching the full per-head K/V tensors in the conventional way.

This is conceptually different from MQA/GQA:

- **MQA:** one shared K/V head;
- **GQA:** a smaller number of shared K/V heads;
- **MLA:** low-rank latent compression of K/V information.

DeepSeek-V2 reports a 93.3% KV-cache reduction relative to its stated comparison architecture. That number is a paper-specific result, not a universal "MLA compression ratio."

RoPE complication

DeepSeek-V2 also uses decoupled rotary positional components because ordinary RoPE interacts with the projection-absorption strategy used by MLA. This is why a simple "MLA caches one tiny vector" summary is incomplete.

Research anchors: [4]

16 — 位置信息

对应原文第 18 页；点击查看相邻原文对照页

RoPE 与缓存的键如何处理

标准旋转位置嵌入（Rotary Position Embeddings，RoPE）在注意力使用点积前，对查询与键施加依赖位置的旋转。普通缓存解码中，保存以供未来复用的历史键，必须与后续查询预期的位置处理保持一致。

从实现角度，可按以下方式理解：为新位置计算新键；按注意力实现的要求应用模型的位置变换；以未来注意力调用预期的形式存储或复用键表示。

不同架构可能改变具体存储内容。MLA 是清楚的例子：与传统 MHA/GQA 模型相比，它的解耦位置分量（Decoupled Positional Component）改变了缓存结构。

不要把“KV 缓存总是存储经过 RoPE 的 K”当成通用定义。缓存表示取决于架构及实现。

研究依据：[4]、[6]。

16 - POSITION INFORMATION

What happens to RoPE and cached keys?

With standard rotary position embeddings, position-dependent rotations are applied to queries and keys before their dot products are used by attention. During ordinary cached decoding, the historical keys stored for future reuse must remain consistent with the positional treatment expected by later queries.

A useful implementation-level mental model is:

- compute the new key for the new position;
- apply the model's positional transformation as required by the attention implementation;
- store/reuse the key representation in the form expected by future attention calls.

Different architectures can change exactly what is stored. MLA is the clearest example: its decoupled positional component changes the cache structure compared with a conventional MHA/GQA model.

Do not hard-code "the KV cache always stores post-RoPE K" as a universal definition. Cache representation is architecture- and implementation-dependent.

Research anchors: [4], [6]

17 — 从单个请求到服务器

对应原文第 19 页；点击查看相邻原文对照页

为什么缓存管理变得困难

单个请求的情况很简单：缓存随词元处理而增长。推理服务（Serving）系统则同时处理多个请求，其提示长度、生成长度、到达时间及完成时间各不相同。

任意时刻，服务器可能同时管理：刚开始的序列；已经缓存数万个词元的序列；完成并释放内存的多个请求；共享前缀的请求；需要不同注意力窗口模式的请求。

这使 KV 缓存从一个张量转变为动态内存管理（Dynamic Memory Management）问题。

PagedAttention 论文指出，碎片化（Fragmentation）及冗余副本是当时推理服务系统中浪费 KV 缓存容量的主要来源，因此提出受虚拟内存（Virtual Memory）启发的分块设计。

研究依据：[5]。

17 - FROM ONE REQUEST TO A SERVER

Why cache management becomes difficult

A single request has a simple story: its cache grows as tokens are processed. A serving system has many requests with different prompt lengths, generation lengths, arrival times, and completion times.

At any moment, the server may need to manage:

- one sequence that has just started;
- another already holding tens of thousands of cached tokens;
- several requests that finish and free memory;
- requests that share prefixes;
- requests that need different attention-window patterns.

This turns KV cache from a tensor into a dynamic memory-management problem.

The PagedAttention paper identified fragmentation and redundant duplication as major sources of wasted KV-cache capacity in contemporary serving systems, motivating a block-based design inspired by virtual memory.

Research anchors: [5]

18 — 碎片化

对应原文第 20 页；点击查看相邻原文对照页

为什么连续分配会浪费容量

如果每个请求都需要一块按其最大可能序列长度预留的大型连续 KV 内存，会产生两类浪费。

预留但未使用的空间（Reserved-but-unused Space）：请求可能预留了最终根本用不到的容量。

外部碎片化（External Fragmentation）：空闲内存分散成若干片段，每片都不足以满足一个较大的连续分配请求，即使总空闲容量足够。

PagedAttention 不要求每条逻辑序列的 KV 状态都位于单个连续物理区域，而是将缓存划分为固定大小的块，并将逻辑块映射到物理块。

分页（Paging）解决内存管理问题。FlashAttention 主要是计算注意力的 IO 感知算法／内核策略（IO-aware Algorithm/Kernel Strategy），两者概念不同。

研究依据：[5]。

18 - FRAGMENTATION

Why contiguous allocation wastes capacity

If every request needs one large contiguous KV allocation sized for its maximum possible sequence length, two kinds of waste can appear:

Reserved-but-unused space

A request may reserve room it never reaches.

External fragmentation

Free memory can exist in pieces that are individually too small to satisfy a large contiguous request even when their total size is sufficient.

The PagedAttention design avoids requiring each logical sequence's KV state to live in one contiguous physical region. Instead it divides cache state into fixed-size blocks and maps logical blocks to physical blocks.

Paging is about memory management. It should not be confused with FlashAttention, which is primarily an IO-aware algorithm/kernel strategy for computing attention.

Research anchors: [5]

19 — PagedAttention

对应原文第 21 页；点击查看相邻原文对照页

逻辑序列与不连续的物理块

分页注意力（PagedAttention）将操作系统中的思想应用于 KV 状态。序列被划分为逻辑词元块（Logical Token Blocks），这些块可存放在任意可用的物理 KV 缓存块中。

原图标签对照：Logical sequence＝逻辑序列，依次为 B0、B1、B2、B3；Physical cache＝物理缓存，依次示意 B2、空闲（free）、B0、B3、空闲、B1。物理顺序无需等于逻辑顺序。

块表（Block Table）告诉注意力实现每个逻辑块位于何处。块可随请求增长而分配，不再需要时释放。

最初的 vLLM 工作还利用块共享（Block Sharing）减少相关序列的重复副本。

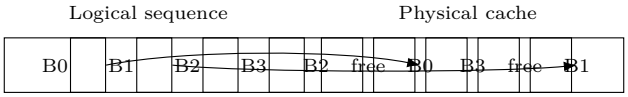
研究依据：[5]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

19 - PAGEDATTENTION

Logical sequence, non-contiguous physical blocks

PagedAttention applies an operating-systems-inspired idea to KV state. A sequence is divided into logical token blocks. Those logical blocks can reside in arbitrary available physical KV-cache blocks.



A block table tells the attention implementation where each logical block lives. Blocks can be allocated as the request grows and freed when no longer needed.

The original vLLM work also used block sharing to reduce duplication for related sequences.

Research anchors: [5]

20 — 块大小

对应原文第 22 页；点击查看相邻原文对照页

分页也有自己的权衡

一个 KV 块对应固定范围的词元位置。较小的块可减少序列尾部的内部浪费，让复用粒度更细。较大的块可减少管理开销，也可能更适合某些内核或内存传输方式。

不存在脱离引擎、硬件、工作负载、模型及内核设计的通用最佳块大小（Block Size）。

现代推理服务系统提供面向块的缓存管理器，但其具体默认值及物理打包方式（Physical Packing）应视为实现细节，而非 KV 缓存定义的一部分。

持久有效的概念是：逻辑词元历史并不等同于单块连续物理内存。原文的不等价符号保留在对照页。

这层分离使运行时能够以块为粒度分配、共享、逐出及移动缓存状态。

研究依据：[5]、[7]。

20 - BLOCK SIZE

Paging has its own trade-offs

A KV block represents a fixed span of token positions. Smaller blocks can reduce internal waste at the tail of a sequence and make reuse more granular. Larger blocks can reduce bookkeeping and may better fit some kernel/memory-transfer choices.

There is no universal optimal block size independent of engine, hardware, workload, model, and kernel design.

Modern serving systems expose block-oriented cache managers, but their exact defaults and physical packing should be treated as implementation details rather than part of the definition of KV caching.

The durable idea

$$\text{logical token history} \quad \neq \quad \text{one contiguous physical allocation}$$

That separation lets the runtime allocate, share, evict, and move cache state at block granularity.

Research anchors: [5], [7]

21 — 前缀缓存

对应原文第 23 页：点击查看相邻原文对照页

跨请求复用预填充状态

普通 KV 缓存是在一个正在执行的请求内复用较早状态。前缀缓存（Prefix Caching）更进一步：如果后来的请求有一个与先前已计算状态匹配、可复用的前缀，服务器就能复用该前缀的缓存 KV 块，而无需重算预填充（Prefill）。

原文所述的当前 vLLM 使用基于前缀及块内容的块哈希（Block Hashes）实现自动前缀缓存。必要时还加入保证正确性的额外输入，例如模型或运行时专有的上下文。

两类受益工作负载：围绕同一长文档或系统提示反复提问；多轮对话中反复把大量历史作为前缀发送。

前缀缓存是对匹配计算前缀的精确状态复用（Exact-state Reuse）。它不是语义相似性搜索，也不是检索增强生成（RAG）。

研究依据：[7]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

21 - PREFIX CACHING

Reuse prefill state across requests

Ordinary KV caching reuses earlier state *within* an ongoing request. Prefix caching goes further: if a later request has a reusable prefix that matches previously computed state, the server can reuse cached KV blocks for that prefix instead of recomputing its prefill.

Current vLLM implements automatic prefix caching with block hashes based on the prefix and block content, plus additional inputs when necessary for correctness (for example, model/runtime-specific context).

Two workloads that benefit

- repeated questions over the same long document or system prompt;
- multi-turn conversations where a large history is repeatedly sent as a prefix.

Prefix caching is exact-state reuse for a matching computational prefix. It is not semantic similarity search and it is not RAG.

Research anchors: [7]

22 — 前缀缓存的限制

对应原文第 24 页；点击查看相邻原文对照页

减少预填充工作，不减少之后的解码工作

前缀缓存可跳过先前处理过的共享前缀的计算。一旦生成超出该前缀，新词元仍须正常解码。

原文引用的 vLLM 文档明确指出：自动前缀缓存减少查询处理／预填充时间，但不减少解码阶段生成新词元所需的时间。

因此，若工作负载主要耗时在生成很长的答案，即使提示前缀可复用，也可能收益很小。

正确性不只要求文本匹配：安全的缓存键（Cache Key）可能需要区分模型身份及配置、适配器（Adapters）、多模态输入（Multimodal Inputs）或其他执行状态，因为这些因素会影响缓存块能否复用。具体要求取决于运行时。

研究依据：[7]。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

22 - PREFIX CACHING LIMITS

It reduces prefill work, not future decode work

Prefix caching can skip computation for a previously processed shared prefix. Once generation moves beyond that prefix, the new tokens still have to be decoded normally.

Current vLLM documentation states this explicitly: automatic prefix caching reduces query-processing/prefill time, but does not reduce the time required to generate new tokens in the decode phase.

That means a workload dominated by a very long generated answer may see little benefit even if its prompt prefix is reusable.

Correctness requires more than matching text

A safe cache key may need to distinguish more than visible token text: model identity and configuration, adapters, multimodal inputs, or other execution state can affect whether a cached block is reusable. Exact requirements are runtime-specific.

Research anchors: [7]

23 — 前缀缓存安全

对应原文第 25 页：点击查看相邻原文对照页

复用可能通过时间差泄露信息

多租户服务（Multi-tenant Service）中，缓存命中（Cache Hits）可能改变延迟（Latency）。如果不相关用户可以共享前缀缓存状态，攻击者可能观察时间差，推断某些内容是否已经缓存。

原文所述的当前 vLLM 提供可选的每请求 cache_salt。这个盐值（Salt）被纳入首个块的缓存标识，使只有共享盐值的请求才能复用这些块，从而建立缓存共享的信任边界（Trust Boundary）。

这一工程原则并不限于某个框架：缓存复用范围应与信任范围一致。

哈希选择也重要：原文引用的 vLLM 文档警告，非密码学哈希（Non-cryptographic Hashing）增加碰撞风险，并可能在共享环境中造成正确性或隐私问题。

研究依据：[7]。

23 - PREFIX CACHING SECURITY

Reuse can reveal information through timing

In a multi-tenant service, cache hits can change latency. If unrelated users are allowed to share prefix-cache state, an attacker may be able to infer whether some content was already cached by observing timing differences.

Current vLLM provides optional per-request `cache_salt`. The salt is incorporated into the first block's cache identity so only requests that share the salt can reuse those blocks, creating a trust boundary for cache sharing.

The same engineering principle is broader than any one framework:

cache reuse domain should match trust domain.

Hash selection also matters: current vLLM documentation warns that non-cryptographic hashing increases collision risk and can create correctness or privacy concerns in shared environments.

Research anchors: [7]

24 — 缓存逐出

对应原文第 26 页：点击查看相邻原文对照页

GPU 缓存满了会怎样

生产环境缓存容量有限。需要新的 K/V 块时，旧的可复用块可能必须离开 GPU 内存。

没有单一通用的逐出策略（Eviction Policy）。示例包括：可复用前缀块的最近最少使用（Least Recently Used，LRU）式逐出；考虑优先级的 LRU 变体；保护正在解码请求所属块的策略；可扩展卸载系统中由应用定义的策略。

原文所述的 vLLM 前缀缓存设计记录了基于 LRU 的释放／逐出路径；TensorRT-LLM 文档描述了可复用块的优先级 LRU。

逐出可复用前缀块不会改变模型正确性，只会影响未来请求能否避免重算该状态。逐出活动序列必需的缓存则不同，必须由运行时协调。

研究依据：[7]、[8]。

24 - EVICTION

What happens when GPU cache is full?

A production cache has finite capacity. When new K/V blocks are needed, old reusable blocks may have to leave GPU memory.

There is no single universal eviction policy. Examples include:

- LRU-style eviction for reusable prefix blocks;
- priority-aware LRU variants;
- policies that protect blocks belonging to actively decoding requests;
- application-defined policies in extensible offloading systems.

Current vLLM prefix-cache design documents an LRU-based free/eviction path. Current TensorRT-LLM documentation describes prioritized LRU for reusable blocks.

Evicting a reusable prefix block does not change model correctness. It changes whether future requests can avoid recomputing that state. Evicting cache required by an active sequence is a different matter and must be coordinated by the runtime.

Research anchors: [7], [8]

25 — 缓存卸载

对应原文第 27 页；点击查看相邻原文对照页

用较慢的内存作为更大的缓存层

GPU 内存紧张时，可把可复用 KV 块移至更大但较慢的存储，需要时再调回。常见层次是：GPU 高带宽内存（HBM）↔ CPU 内存 ↔ 次级存储／网络层。

原文所述的当前 vLLM 支持 CPU KV 卸载（Offloading）及多层卸载设计，次级存储层经 CPU 内存中转。TensorRT-LLM 可将可复用块卸载到主机内存（Host Memory），并提供用于外部 KV 缓存存储或传输的连接器（Connectors）。

权衡：卸载扩大有效缓存容量（Cache Capacity），但引入数据移动（Data Movement）。是否有益取决于复用概率、传输带宽／延迟、与计算的重叠程度，以及选择重算状态的成本。

“卸载到 CPU 以节省 GPU 内存”并不完整。真正的选择是传输缓存状态，还是重新计算状态。

研究依据：[8]、[9]。

25 - OFFLOADING

Use slower memory as a larger cache tier

When GPU memory is scarce, reusable KV blocks can be moved to larger but slower storage and promoted back when needed.

A common hierarchy is

$$\text{GPU HBM} \leftrightarrow \text{CPU memory} \leftrightarrow \text{secondary storage/network tier.}$$

Current vLLM supports CPU KV offloading and a multi-tier offloading design in which secondary tiers are staged through CPU memory. Current TensorRT-LLM can offload reusable blocks to host memory and exposes connectors for external KV-cache storage or transfer.

The trade-off

Offloading expands effective cache capacity, but introduces data movement. Whether it helps depends on reuse probability, transfer bandwidth/latency, overlap with compute, and the cost of recomputing the state instead.

"Offload to CPU to save GPU memory" is incomplete. The real decision is: transfer cached state or recompute it?

Research anchors: [8], [9]

26 — KV 缓存量化

对应原文第 28 页；点击查看相邻原文对照页

用更少的位存储 K/V

降低缓存容量及带宽压力的另一条路径，是以较低精度（Precision）存储 K/V 激活值（Activations）。

概念上，若张量有效载荷从 16 位存储变为理想的 8 位存储，元素存储部分将减半。实际实现还需要缩放因子（Scales）、对齐（Alignment）、元数据及后端专有布局，因此实际内存节省不一定恰好等于理想位宽比例。

原文所述的 vLLM 支持多种 KV 缓存存储数据类型（Dtypes），包括受支持硬件／后端上的 FP8 变体，并记录了量化缩放因子的校准（Calibration）路径。

KVQuant 等研究系统探索更低位宽的 KV 表示，但其准确率（Accuracy）及吞吐量（Throughput）结果取决于具体方法、模型、硬件及基准测试（Benchmark）。

研究依据：[10]、[11]。

26 - KV-CACHE QUANTIZATION

Store K/V with fewer bits

Another way to reduce cache capacity and bandwidth pressure is to store K/V activations at lower precision.

Conceptually, if the tensor payload moved from 16-bit storage to ideal 8-bit storage, the element-storage component would halve. Real implementations also require scales, alignment, metadata, and backend-specific layouts, so practical memory savings need not equal the ideal bit-width ratio exactly.

Current vLLM supports multiple KV-cache storage dtypes, including FP8 variants on supported hardware/backends. It also documents calibration pathways for quantization scales.

Research systems such as KVQuant study even lower-bit KV representations, but their reported accuracy/throughput results are method-, model-, hardware-, and benchmark-specific.

Research anchors: [10], [11]

27 — 量化并非没有代价

对应原文第 29 页：点击查看相邻原文对照页

压缩会改变数值状态

与 PagedAttention 的内存放置或精确前缀复用不同，KV 缓存量化（Quantization）改变缓存张量的数值表示（Numerical Representation），将量化误差（Quantization Error）带入未来的注意力计算。

工程问题由此变为：节省的内存／带宽，与模型质量及内核成本之间如何权衡。

合适精度可能取决于：模型架构及各层敏感性；键／值激活分布；上下文长度；量化粒度（Granularity）及缩放因子；硬件支持及可用内核（Kernels）。

原文所述的 vLLM 甚至允许选定层或注意力类型保持不量化，而其他层使用较低精度 KV 数据类型。这体现了各层敏感性不必相同。

研究依据：[10]、[11]。

27 - QUANTIZATION IS NOT FREE

Compression changes numerical state

Unlike PagedAttention's memory placement or exact prefix reuse, KV-cache quantization changes the numerical representation of cached tensors.

That introduces quantization error into future attention computations. The engineering question becomes:

$$\text{memory/bandwidth saved} \quad \text{vs} \quad \text{quality and kernel cost.}$$

The right precision can depend on:

- model architecture and layer sensitivity;
- key/value activation distributions;
- context length;
- quantization granularity and scales;
- hardware support and available kernels.

Current vLLM even allows selected layers or attention types to remain unquantized while others use a lower-precision KV dtype, reflecting the fact that sensitivity need not be uniform.

Research anchors: [10], [11]

28 — 常见误解

对应原文第 30 页；点击查看相邻原文对照页

哪些简略说法会造成错误理解

“KV 缓存是模型的记忆。”不准确。它是注意力层从词元序列派生的临时推理状态（Inference State）。

“KV 缓存使解码成为常数时间。”不准确。它避免重算历史 K/V 状态；新查询的稠密注意力仍需与不断增长的历史交互。

“MQA/GQA 减少查询头。”不准确。其定义性改变是相对于查询头的 K/V 头数量及共享方式。

“PagedAttention 就是 FlashAttention。”不准确。前者处理 KV 缓存内存管理；后者处理高效注意力计算及内存流量（Memory Traffic）。

“前缀缓存会加快每个生成词元。”不准确。它可跳过可复用前缀的预填充，却不消除新词元的常规解码工作。

“KV 量化是无损压缩。”通常不成立。较低精度表示引入数值近似（Numerical Approximation）。

研究依据：[2]、[5]、[7]、[10]。

28 - COMMON MISCONCEPTIONS

Shortcuts that create the wrong mental model

“KV cache is the model’s memory.”
No. It is temporary inference state derived from a token sequence at attention layers.

“KV caching makes decoding constant-time.”
No. It avoids recomputing historical K/V states; dense attention for the new query still interacts with an expanding history.

“MQA/GQA reduce query heads.”
No. Their defining change is the number/sharing of K/V heads relative to query heads.

“PagedAttention is FlashAttention.”
No. PagedAttention addresses KV-cache memory management; FlashAttention addresses efficient attention computation/memory traffic.

“Prefix caching speeds every generated token.”
No. It can skip reusable prefix prefill; it does not remove normal decode work for new tokens.

“KV quantization is lossless compression.”
Generally no. Lower-precision representation introduces numerical approximation.

Research anchors: [2], [5], [7], [10]

29 — 生产环境中的理解框架

对应原文第 31 页；点击查看相邻原文对照页

评估任何 KV 缓存设计的五个问题

1. 存储什么？保存哪些 K/V 或潜在状态，位于哪些层，使用什么数据类型／布局？

2. 每个词元产生多少状态？追踪 L、H_KV、D_h、字节数、注意力窗口规则及分片（Sharding）。

3. 状态存放在哪里？连续 GPU 张量、分页块、CPU 卸载、远程缓存，还是多个存储层？

4. 什么时候可以复用？请求内部、匹配前缀之间、实例之间，还是完全无法复用？

5. 瓶颈是什么？GPU 容量、内存带宽（Memory Bandwidth）、传输延迟（Latency）、重计算（Recomputation）、碎片化（Fragmentation），还是调度器压力？

这些问题将 KV 缓存的数学机制，与围绕它构建的推理服务（Serving）系统区分开来。

研究依据：[5]、[7]、[8]、[9]。

29 - A PRODUCTION MENTAL MODEL

Five questions to ask about any KV-cache design

1. **What is stored?**
   Which K/V or latent states, at which layers, in what dtype/layout?
2. **How much state is created per token?**
   Track L, H_{KV}, D_h, bytes, attention-window rules, and sharding.
3. **Where does the state live?**
   Contiguous GPU tensors, paged blocks, CPU offload, remote cache, or multiple tiers?
4. **When can it be reused?**
   Within a request, across matching prefixes, across instances, or not at all?
5. **What is the bottleneck?**
   GPU capacity, memory bandwidth, transfer latency, recomputation, fragmentation, or scheduler pressure?

These questions separate the mathematical KV-cache mechanism from the serving system built around it.

Research anchors: [5], [7], [8], [9]

30 — 完整理解框架

对应原文第 32 页；点击查看相邻原文对照页

用一条逻辑链理解 KV 缓存

因果 Transformer 反复根据已有词元生成新词元。追加未来词元时，过去的注意力层状态不变，所以模型可以保留其键和值，而无需重算。

传统缓存的张量有效载荷大致按 2 × L × T × H_KV × D_h × b 增长；完整公式保留在原文对照页。

因此，长上下文及并发请求成本很高。MQA/GQA 减少键值头数；MLA 压缩缓存表示；较低精度存储减少每元素字节数。

在大规模推理服务中，动态序列长度带来分配及复用问题。分块缓存管理把逻辑词元历史与物理内存放置分开。前缀缓存跨兼容请求复用已计算的预填充状态；逐出（Eviction）及卸载（Offloading）决定哪些可复用块留在 GPU 附近；量化（Quantization）以数值精度（Numerical Precision）换取容量及带宽节省。

KV 缓存同时是因果注意力（Causal Attention）的结果、模型架构的设计约束，以及运行时内存管理问题。理解这三个层次，才能真正运用这个概念。

研究依据：[1]—[11]。

30 - THE COMPLETE MENTAL MODEL

KV cache in one chain

A causal Transformer repeatedly generates a new token from prior tokens. Past attention-layer states do not change when future tokens are appended, so the model can retain their keys and values instead of recomputing them.

For a conventional cache, tensor payload grows roughly as

$$2LTH_{KV}D_h b.$$

That makes context length and concurrent requests expensive. MQA/GQA reduce the number of KV heads; MLA compresses the cached representation; lower-precision storage reduces bytes per element.

At serving scale, dynamic sequence lengths create allocation and reuse problems. Block-based cache management separates logical token history from physical memory placement. Prefix caching reuses already-computed prefill state across compatible requests. Eviction and offloading decide which reusable blocks remain near the GPU. Quantization trades numerical precision for capacity/bandwidth savings.

The KV cache is simultaneously a consequence of causal attention, a model-architecture design constraint, and a runtime memory-management problem. Understanding all three levels is what makes the concept useful.

Research anchors: [1]-[11]

参考文献

对应原文第 33 页；点击查看相邻原文对照页

主要论文及原文引用的实现资料

[1] A. Vaswani 等。《注意力就是你所需要的一切》（Attention Is All You Need）。NeurIPS 2017。arXiv: 1706.03762。

[2] N. Shazeer。《快速 Transformer 解码：一个写入头就够了》（Fast Transformer Decoding: One Write-Head is All You Need）。arXiv: 1911.02150，2019。

[3] J. Ainslie 等。《GQA：从多头检查点训练广义多查询 Transformer 模型》（GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints）。EMNLP 2023。arXiv: 2305.13245。

[4] DeepSeek-AI 等。《DeepSeek-V2：强大、经济且高效的混合专家语言模型》（DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model）。arXiv: 2405.04434，2024。

[5] W. Kwon 等。《使用 PagedAttention 高效管理大语言模型推理服务的内存》（Efficient Memory Management for Large Language Model Serving with PagedAttention）。SOSP 2023。DOI: 10.1145/3600006.3613165；arXiv: 2309.06180。

[6] NVIDIA Transformer Engine 文档。PyTorch 注意力／推理参数（Attention/Inference Parameters）；解码时的 K/V 缓存（K/V Caching During Decoding）。原作者查阅于 2026 年 9 月。

[7] vLLM 文档。自动前缀缓存（Automatic Prefix Caching）、缓存配置（Cache Configuration）及前缀缓存隔离／安全（Prefix-cache Isolation/Security）。原作者查阅当前文档于 2026 年 9 月。

[8] NVIDIA TensorRT-LLM 文档。KV 缓存系统（KV Cache System）、复用（Reuse）、优先级逐出（Prioritized Eviction）及主机卸载（Host Offloading）。原作者查阅当前文档于 2026 年 9 月。

[9] vLLM 文档。KV 卸载使用指南（KV Offloading Usage Guide）及多层卸载 API（Multi-tier Offloading APIs）。原作者查阅当前文档于 2026 年 9 月。

[10] vLLM 文档。量化 KV 缓存（Quantized KV Cache）及 CacheConfig 存储数据类型选项（Storage Dtype Options）。原作者查阅当前文档于 2026 年 9 月。

[11] C. Hooper 等。《KVQuant：通过 KV 缓存量化迈向一千万上下文长度的大语言模型推理》（KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization）。arXiv: 2401.18079，2024。

REFERENCES

Primary papers and current implementation sources

[1] A. Vaswani et al. *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762.

[2] N. Shazeer. *Fast Transformer Decoding: One Write-Head is All You Need.* arXiv:1911.02150, 2019.

[3] J. Ainslie et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.* EMNLP 2023. arXiv:2305.13245.

[4] DeepSeek-AI et al. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.* arXiv:2405.04434, 2024.

[5] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention.* SOSP 2023. DOI:10.1145/3600006.3613165; arXiv:2309.06180.

[6] NVIDIA Transformer Engine documentation. PyTorch attention/inference parameters; K/V caching during decoding. Consulted September 2026.

[7] vLLM documentation. Automatic Prefix Caching, cache configuration, and prefix-cache isolation/security. Current documentation consulted September 2026.

[8] NVIDIA TensorRT-LLM documentation. KV Cache System, reuse, prioritized eviction, and host offloading. Current documentation consulted September 2026.

[9] vLLM documentation. KV Offloading Usage Guide and multi-tier offloading APIs. Current documentation consulted September 2026.

[10] vLLM documentation. Quantized KV Cache and CacheConfig storage dtype options. Current documentation consulted September 2026.

[11] C. Hooper et al. *KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization.* arXiv:2401.18079, 2024.

@techNmak